

مستندات فایل‌های **MSLesSeg.py**، **dataset_utils.py** و **test.py**

کلاس Dataset و DataLoader در چارچوب MS-SAM-LESS

پایان‌نامه کارشناسی ارشد – مهندسی کامپیوتر، گرایش هوش مصنوعی

۶ مهر ۱۴۰۵

فهرست مطالب

۲	۱	مقدمه
۲	۲	فایل MSLesSeg.py
۲	۱.۲	بخش ۱: <code>import</code> ها
۲	۲.۲	بخش ۲: سازنده‌ی کلاس <code>MSLesSeg</code>
۳	۳.۲	بخش ۳: متد <code>_generate_data_list</code>
۴	۴.۲	بخش ۴: تابع <code>get_mslesseg_dataset</code>
۵	۵.۲	بخش ۵: تابع <code>get_test_mslesseg_dataset</code>
۶	۳	فایل dataset_utils.py
۸	۴	بخش تست در test.py
۸	۵	جریان کامل داده (End-to-End)
۸	۶	نکات و مشاهدات کلیدی
۸	۷	ارتباط با پروژه‌ی Polyp
۸	۸	جمع‌بندی

۱ مقدمه

در این سند، سه فایل مرتبط با بارگذاری داده در چارچوب MS-SAM-LESS بررسی می‌شود:

۱. **MSLesSeg.py**: تعریف کلاس Dataset که از CacheDataset کتابخانه MONAI ارث‌بری می‌کند.

۲. **dataset_utils.py**: ساخت DataLoader برای آموزش و اعتبارسنجی.

۳. **بخش تست در test.py**: ساخت DataLoader برای تست.

این سه فایل به صورت زنجیره‌ای کار می‌کنند: از فایل JSON شروع می‌کنند، داده را به tensor تبدیل می‌کنند و در نهایت batchهای آماده برای مدل تولید می‌کنند.

جریان کلی داده: `split.json` → `MSLesSeg` → `CacheDataset` + `transforms` → `DataLoader` مدل.

۲ فایل MSLesSeg.py

۱.۲ بخش ۱: `import`

```
1 import sys
2 import json
3 from monai.data import CacheDataset
4 from dataset.tfs import get_mslesseg_transform
```

جدول ۱: کاربرد هر `import` در `MSLesSeg.py`

کاربرد	import
برای دسترسی به <code>sys.maxsize</code> (عدد بزرگ برای cache)	<code>sys</code>
برای خواندن فایل <code>split.json</code>	<code>json</code>
کلاس پایه از MONAI که داده‌ها را در حافظه cache می‌کند	<code>CacheDataset</code>
تابعی که transformهای <code>train/val</code> را از <code>tfs.py</code> می‌گیرد	<code>get_mslesseg_transform</code>

۲.۲ بخش ۲: سازندهی کلاس `MSLesSeg`

```
1 class MSLesSeg(CacheDataset):
2     def __init__(self, root_dir, split_path, mask, section, num_fold,
3                 transforms, seed=100, cache_num=sys.maxsize,
4                 cache_rate=0., num_workers=0):
5         self.mask = mask
6         self.section = section
7         self.num_fold = num_fold
8         data = self._generate_data_list(split_path)
9
10        super().__init__(data, transforms, cache_num=cache_num,
11                        cache_rate=cache_rate, num_workers=num_workers)
```

توضیح:

کلاس `MSLesSeg` از `CacheDataset` ارث‌بری می‌کند. داده‌ها را یک بار در ابتدا بارگذاری و cache می‌کند و بعد در هر epoch از حافظه می‌خواند. این کار سرعت آموزش را به‌طور چشمگیری افزایش می‌دهد، مخصوصاً برای حجم‌های سنگین MRI. پارامترهای سازنده: سه خط ذخیره‌سازی:

جدول ۲: پارامترهای سازنده‌ی MSLesSeg

پارامتر	توضیح
root_dir	پوشه‌ی ریشه‌ی دیتاست (در این کد استفاده نمی‌شود!)
split_path	مسیر فایل split.json
mask	نام کلید ماسک در JSON (مثلاً "mask")
section	یکی از 'train'، 'validation' یا 'test'
num_fold	شماره fold (مثلاً ۰ برای fold0)
transforms	pipeline تبدیلات از tf.py
seed	seed برای تکرارپذیری (ذخیره می‌شود ولی استفاده نمی‌شود)
cache_num	حداکثر تعداد آیت‌ها برای cache (پیش‌فرض: sys.maxsize)
cache_rate	نسبت داده برای cache (۰ = هیچ، ۱ = همه)
num_workers	تعداد thread برای بارگذاری موازی

```

۱ self.mask = mask
۲ self.section = section
۳ self.num_fold = num_fold

```

این سه مقدار به عنوان attribute ذخیره می‌شوند تا بعداً در `_generate_data_list` استفاده شوند.
خط کلیدی:

```

۱ data = self._generate_data_list(split_path)

```

تابع `_generate_data_list` را صدا می‌زند که لیست نمونه‌ها را از JSON می‌خواند.
فراخوانی سازنده‌ی پدر:

```

۱ super().__init__(data, transforms, cache_num=cache_num,
۲                  cache_rate=cache_rate, num_workers=num_workers)

```

CacheDataset را با لیست داده، transform‌ها و تنظیمات cache مقداردهی می‌کند.

نکته: پارامتر `root_dir` و `seed` در سازنده گرفته می‌شوند ولی در هیچ جای کد استفاده نمی‌شوند. `root_dir` چون مسیرها در JSON کامل هستند و `seed` احتمالاً برای توسعه‌ی آینده ذخیره شده است.

۳.۲ بخش ۳: متد `_generate_data_list`

```

۱ def _generate_data_list(self, split_path):
۲     with open(split_path) as fp:
۳         data = json.load(fp)

```

توضیح:

- فایل JSON را باز می‌کند و به دیکشنری پایتون تبدیل می‌کند.
- fp نام دلخواه برای file pointer است.

مثال: اگر split.json به این شکل باشد:

```

{
  "fold0": {
    "train": [{"image": "...", "mask": "..."}, ...],

```

```

    "val": [{"image": "...", "mask": "..."}, ...]
  },
  "test": [{"image": "...", "mask": "..."}, ...]
}

```

آنگاه data یک دیکشنری پایتون با همان ساختار می‌شود.

```

1 if self.section == 'validation':
2     datalist = data[f'fold{self.num_fold}']['val']
3 elif self.section == 'train':
4     datalist = data[f'fold{self.num_fold}']['train']
5 elif self.section == 'test':
6     datalist = data['test']
7 else:
8     raise ValueError(
9         f"Unsupported section: {self.section}, "
10        f"available options are ['train', 'validation', 'test'].")
11 )
12 return datalist

```

توضیح:

- بر اساس مقدار self.section، لیست مربوطه استخراج می‌شود.
- 'f'fold{self.num_fold}' با f-string ساخته می‌شود → مثلاً اگر num_fold=0 باشد، می‌شود 'fold0'.
- اگر section مقدار نامعتبر باشد، Exception پرتاب می‌کند.

مثال:

کلید استخراج شده از JSON	num_fold	section
data['fold0']['train']	◦	'train'
data['fold0']['val']	◦	'validation'
data['test']	◦	'test'

نکته طراحی: 'validation' در این تابع به کلید 'val' در JSON نگاشت می‌شود. این یک تصمیم طراحی است تا API کاربرپسندتر باشد.

۴.۲ بخش ۴: تابع get_mslesseg_dataset

```

1 def get_mslesseg_dataset(args, sam_image_dim=None):
2     split_path = args['split_path']
3     mask = args['mask_key']
4     num_fold = args['n_fold']

```

توضیح: استخراج مسیر split، نام کلید ماسک و شماره fold از آرگومان‌های برنامه.

```

1 if args['sam_zeroshot']:
2     get_bbox = True
3 else:
4     get_bbox = False

```

توضیح: اگر حالت **zero-shot SAM** فعال باشد (sam_zeroshot=True)، فلگ get_bbox=True می‌شود. این احتمالاً به transform می‌گوید که bounding-box تولید کنند (چون SAM در حالت zero-shot به prompt نیاز دارد).

```

1 transform_train, transform_val_test = get_mslesseg_transform(
2     args, sam_image_dim, get_bbox=get_bbox
3 )

```

توضیح: از tfs.py دو pipeline می‌گیریم: یکی برای train و یکی برای val/test.

```

1 ds_train = MSLesSeg(args['data_dir'], split_path, mask, section='train',
2                     num_fold=num_fold, transforms=transform_train,
3                     cache_rate=args['cache_rate'])
4 ds_val = MSLesSeg(args['data_dir'], split_path, mask, section='validation'
5                 ,
6                 num_fold=num_fold, transforms=transform_val_test,
7                 cache_rate=args['cache_rate'])
8 return ds_train, ds_val

```

توضیح:

- دو شیء MSLesSeg می‌سازد: یکی برای آموزش و یکی برای اعتبارسنجی.
- هر دو از یک split_path می‌خوانند ولی section متفاوت دارند.
- cache_rate از args خوانده می‌شود (اگر ° باشد، هیچ چیزی cache نمی‌شود؛ اگر ۱ باشد، همه cache می‌شود).

مثال: اگر args به این شکل باشد:

```

args = {
    'split_path': 'data/split.json',
    'mask_key': 'mask',
    'n_fold': 0,
    'sam_zeroshot': False,
    'data_dir': 'data',
    'cache_rate': 1.0,
}

```

آنگاه:

- ds_train روی data['fold0']['train'] کار می‌کند.
- ds_val روی data['fold0']['val'] کار می‌کند.

۵.۲ بخش ۵: تابع get_test_mslesseg_dataset

```

1 def get_test_mslesseg_dataset(args, sam_image_dim=None):
2     split_path = args['split_path']
3     mask = args['mask_key']
4     num_fold = args['n_fold']
5
6     if args['sam_zeroshot']:
7         get_bbox = True
8     else:
9         get_bbox = False
10
11     transform_train, transform_val_test = get_mslesseg_transform(
12         args, sam_image_dim, get_bbox=get_bbox
13     )
14
15     ds_test = MSLesSeg(args['data_dir'], split_path, mask, section='test',
16                     num_fold=num_fold, transforms=transform_val_test,
17                     cache_rate=args['cache_rate'])
18     return ds_test

```

توضیح:

- بسیار شبیه به `get_mslesseg_dataset` است، ولی فقط یک `dataset` برمی‌گرداند: `ds_test`.
- از `transform_val_test` استفاده می‌کند (بدون `augmentation`).
- `section='test'` باعث می‌شود `_generate_data_list` کلید `data['test']` را بخواند.

نکته‌ی مهم: در این تابع، `num_fold` استخراج می‌شود ولی در بخش `test` بی‌استفاده می‌ماند، چون `test` در سطح ریشه است و به `fold` وابسته نیست.

۳ فایل `dataset_utils.py`

```
1 import torch
2 from dataset.MSLesSeg import get_mslesseg_dataset
3
4 def get_dataset(args, transform, img_dim):
5     if args['task'] == 'mslesseg':
6         trainset, testset = get_mslesseg_dataset(args, sam_image_dim=img_dim)
7         ds = torch.utils.data.DataLoader(
8             trainset, batch_size=int(args['Batch_size']), shuffle=True,
9             num_workers=int(args['nW']), drop_last=True
10        )
11        ds_val = torch.utils.data.DataLoader(
12            testset, batch_size=1, shuffle=False,
13            num_workers=int(args['nW_eval']), drop_last=False
14        )
15        return ds, ds_val
```

تحلیل خط به خط:

```
1 import torch
2 from dataset.MSLesSeg import get_mslesseg_dataset
```

- `DataLoader` برای `torch`.
- تابع `get_mslesseg_dataset` را از فایل `MSLesSeg.py` می‌آورد.

```
1 def get_dataset(args, transform, img_dim):
```

- ورودی‌های تابع:

- آرگومان‌های برنامه؛
- یک `transform` که استفاده نمی‌شود، زیرا در `get_mslesseg_dataset` ساخته می‌شود؛
- ابعاد تصویر `SAM`.

```
1 if args['task'] == 'mslesseg':
2     trainset, testset = get_mslesseg_dataset(args, sam_image_dim=img_dim)
```

- اگر تسک فعلی `mslesseg` باشد، دو `dataset` (`val` و `train`) از `get_mslesseg_dataset` می‌گیرد.

نکته‌ی گمراه‌کننده: متغیر testset نام‌گذاری بدی است! این در واقع مجموعه‌ی اعتبارسنجی (val) است، نه test. احتمالاً در پروژه‌ی اصلی اسمش را عوض نکرده‌اند.

```
1 ds = torch.utils.data.DataLoader(  
2     trainset, batch_size=int(args['Batch_size']), shuffle=True,  
3     num_workers=int(args['nW']), drop_last=True  
4 )
```

• DataLoader برای آموزش:

- batch_size = args['Batch_size'] (مثلاً ۲ یا ۴).
- shuffle=True (ترتیب تصادفی در هر epoch).
- drop_last=True (batch آخر اگر ناقص باشد، حذف می‌شود — مفید برای BatchNorm).

```
1 ds_val = torch.utils.data.DataLoader(  
2     testset, batch_size=1, shuffle=False,  
3     num_workers=int(args['nW_eval']), drop_last=False  
4 )
```

• DataLoader برای اعتبارسنجی:

- batch_size=1 (چون در ارزیابی پزشکی، هر نمونه جدا پردازش می‌شود).
- shuffle=False (ترتیب ثابت برای تکرارپذیری).
- drop_last=False (همه‌ی نمونه‌ها ارزیابی می‌شوند).

```
1 return ds, ds_val
```

• برمی‌گرداند: (train_loader, val_loader).

مثال کامل:

```
args = {  
    'task': 'mslesseg',  
    'Batch_size': 2,  
    'nW': 4,  
    'nW_eval': 2,  
    'split_path': 'data/split.json',  
    'mask_key': 'mask',  
    'n_fold': 0,  
    'sam_zeroshot': False,  
    'data_dir': 'data',  
    'cache_rate': 1.0,  
}  
train_loader, val_loader = get_dataset(args, transform=None, img_dim=1024)
```

حالا می‌توانی:

```
for batch in train_loader:  
    images = batch['image'] # shape: (2, 1, 1024, 1024, 30)  
    masks = batch['mask']
```

۴ بخش تست در test.py

```
۱ if args['task'] == 'mslesseg':
۲     testset = get_test_mslesseg_dataset(args, sam_image_dim=img_dim)
۳
۴ ds_test = torch.utils.data.DataLoader(
۵     testset, batch_size=1, shuffle=False,
۶     num_workers=int(args['nW_eval']), drop_last=False
۷ )
```

تحلیل خط به خط:

```
۱ if args['task'] == 'mslesseg':
۲     testset = get_test_mslesseg_dataset(args, sam_image_dim=img_dim)
```

- اگر تسک mslesseg باشد، dataset تست را می‌سازد.
- testset این بار واقعاً test است (برخلاف dataset_utils.py که testset در واقع val بود).

```
۱ ds_test = torch.utils.data.DataLoader(
۲     testset, batch_size=1, shuffle=False,
۳     num_workers=int(args['nW_eval']), drop_last=False
۴ )
```

- DataLoader برای تست:
 - batch_size=1 (هر نمونه جدا).
 - shuffle=False
 - drop_last=False

نکته‌ی مهم: در test.py فقط یک ds_test ساخته می‌شود، در حالی که در پروژه‌ی polyp ما پنج دیتاست تست داریم. اینجا جایی است که باید شخصی‌سازی کنیم.

۵ جریان کامل داده (End-to-End)

۶ نکات و مشاهدات کلیدی

هشدار حافظه: CacheDataset کل داده را در RAM نگه می‌دارد. برای MSLesSeg کوچک (۱۱۵ حجم) این مشکلی نیست، ولی برای polyp با ۱۴۵۰ تصویر PNG در ابعاد بزرگ ممکن است حافظه‌بر باشد. در این حالت cache_rate را کاهش ده یا از Dataset معمولی استفاده کن.

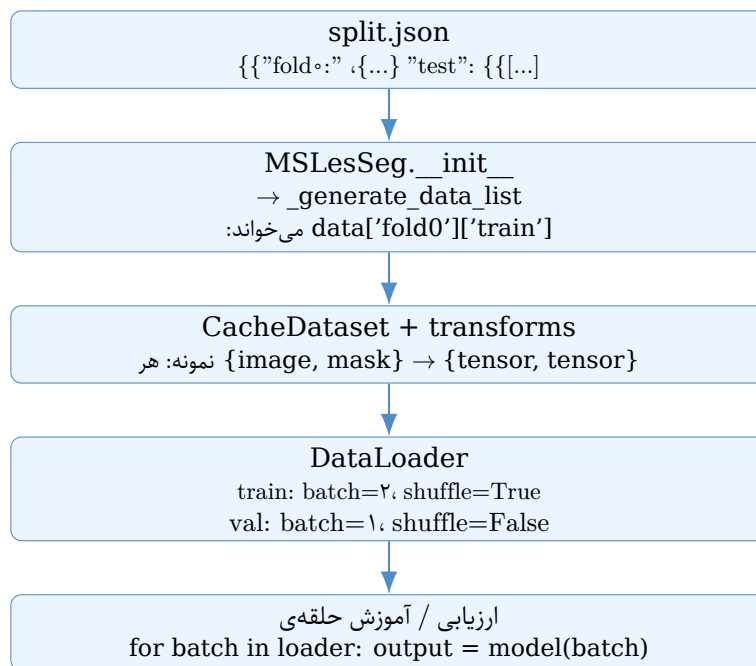
۷ ارتباط با پروژه‌ی Polyp

۸ جمع‌بندی

سه فایل به صورت زنجیره‌ای کار می‌کنند:

۱. **MSLesSeg.py**: Dataset را می‌سازد و از JSON می‌خواند.

۲. **dataset_utils.py**: DataLoader برای آموزش و اعتبارسنجی می‌سازد.



شکل ۱: جریان کامل داده از فایل JSON تا مدل

۳. **test.py**: DataLoader برای تست می‌سازد.

جریان کلی: split.json → MSLesSeg → CacheDataset + transforms → DataLoader → مدل.
 برای polyp، لایه‌ی MSLesSeg باید بازنویسی شود (مثلاً PolypDataset)، ولی لایه‌ی DataLoader تقریباً بدون تغییر باقی می‌ماند.

جدول ۳: مشاهدات مهم در کد

#	نکته
۱	root_dir در سازنده ذخیره نمی‌شود و استفاده نمی‌شود
۲	seed ذخیره می‌شود ولی استفاده نمی‌شود
۳	'validation' به کلید 'val' در JSON نگاشت می‌شود
۴	در dataset_utils.py، متغیر testset در واقع val است (نام‌گذاری همراه‌کننده)
۵	در test.py، testset واقعاً test است
۶	CacheDataset تمام داده را در RAM نگه می‌دارد
۷	num_fold در get_test_mslesseg_dataset گرفته می‌شود ولی استفاده نمی‌شود
۸	پارامتر transform در get_dataset استفاده نمی‌شود

جدول ۴: تغییرات لازم برای انتقال به polyp

بخش	تغییر لازم
MSLesSeg	تبدیل به PolypDataset با پشتیبانی از کلیدهای distribution و dataset_origin
_generate_data_list	ساده‌تر می‌شود چون test یک لیست تخت است
get_mslesseg_dataset	تبدیل به get_polyp_dataset
get_test_mslesseg_dataset	باید بتواند بر اساس dataset_origin یا distribution فیلتر کند
dataset_utils.py	بدون تغییر اساسی
test.py	باید حلقه‌ای روی زیر-dataset تست داشته باشد